Estadística con Python

Tipos de Datos en Python - Calculadora

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba



Tabla de contenidos I

1. Objetivos de la Sesión
2. Tipos de datos: importancia
3. Datos Numéricos
4. Python como calculadora
5. Funciones y métodos aplicables a tuplas
6. Operaciones con tuplas



Objetivos de la Sesión



Objetivos de la Sesión

- Comprender la naturaleza y utilidad de los diferentes tipos de datos en Python.
- Diferenciar entre datos numéricos (enteros y flotantes), cadenas de texto y valores lógicos.
- Reconocer estructuras de datos introductorias relevantes para el análisis de datos.
- Aplicar funciones nativas de Python para verificar y convertir tipos de datos.



Tipos de datos: importancia



Tipos de datos: importancia

- En estadística y programación, la forma en que el computador almacena la información dicta qué operaciones matemáticas o lógicas podemos realizar.
- No podemos calcular la media de un texto, ni concatenar matemáticamente dos números lógicos sin una conversión previa.
- Python es un lenguaje de tipado dinámico, lo que significa que el tipo de dato se asigna automáticamente durante la ejecución.



Datos Numéricos



Datos Numéricos

Python maneja los números principalmente de dos maneras fundamentales para nuestros cálculos.



Enteros (int)

Representan números completos sin parte decimal. Son útiles para contar frecuencias, iteraciones, o tamaños de muestra (n).

```
# Ejemplo de Integer
tamaño_muestra = 150
print(tamaño_muestra)
```

```
150
```

```
print(type(tamaño_muestra))
```

```
<class 'int'>
```



Flotantes (floats)

Representan números reales (con parte decimal). Son el estándar para variables continuas, probabilidades, promedios y estadísticos de prueba.

```
# Ejemplo de Float
media_muestral = 45.89
p_valor = 0.035
print(media_muestral)
```

```
45.89
```

```
print(type(media_muestral))
```

```
<class 'float'>
```



Datos de Texto o Caracteres (`str`)

Conocidos como `strings`, almacenan texto. En el modelado estadístico, a menudo usamos `strings` para etiquetas, nombres de variables categóricas o descripciones cualitativas. Se delimitan con comillas simples o dobles.

```
# Ejemplo de String
categoria_tratamiento = "Tratamiento A"
print(categoria_tratamiento)
```

```
Tratamiento A
```

```
print(type(categoria_tratamiento))
```

```
<class 'str'>
```



Datos Lógicos o Booleanos (`bool`)

- Solo pueden tomar dos valores: `True` (Verdadero) o `False` (Falso).
- Son la base del control de flujo y la evaluación de condiciones (por ejemplo, si un valor supera un umbral crítico).
- En el fondo, Python evalúa True como 1 y False como 0.

```
# Ejemplo de Booleano
hipotesis_rechazada = True
print(hipotesis_rechazada)
```

```
True
```

```
print(type(hipotesis_rechazada))
```

```
<class 'bool'>
```

```
# Evaluación de una condición que retorna un booleano
es_significativo = p_valor < 0.05
print("¿Es estadísticamente significativo?:", es_significativo)
```



Ejemplo de bool

```
# Evaluación de una condición que retorna un booleano
es_significativo = p_valor < 0.05
print("¿Es estadísticamente significativo?:", es_significativo)
```

```
¿Es estadísticamente significativo?: True
```



Datos Especiales: Ausencia de Valor (NoneType)

- En la limpieza de datos, es común encontrar valores perdidos (`Missing values`).
- En Python puro, esto suele representarse con `None`.

```
# Ejemplo de None
dato_faltante = None
print(type(dato_faltante))
```

```
<class 'NoneType'>
```



Conversión de Tipos (Casting)

A veces necesitamos transformar un tipo de dato en otro, por ejemplo, convertir un texto numérico recolectado de un formulario web a un entero.

```
# De string a entero
texto_numero = "120"
numero_real = int(texto_numero)
print(type(numero_real))
```

```
<class 'int'>
```

```
# De float a int (¡Cuidado: se pierde la parte decimal!)
valor_estimado = 4.95
entero_truncado = int(valor_estimado)
print(entero_truncado)
```

```
4
```



Ejercicio Práctico en Clase

- Crear una variable llamada `edad` y asígnenle su edad en años (¿Qué tipo de dato es?).
- Crear una variable estatura en metros (ej. 1.75).
- Crear una variable `nombre_estudiante` con su nombre completo.
- Evaluar de forma lógica (Booleana) si la edad es mayor a 18 y guardar el resultado en una variable llamada `es_mayor_edad`.
- Usen la función `print()` para mostrar todas las variables y la función `type()` para verificar que usaron el tipo correcto.



Ejemplo de aplicación

- Escribir un programa es `python` que capture dos números `a` y `b` y regrese la suma `a+b`
- *Ayuda:* para capturar un valor desde el teclado use la función `input()`



Python como calculadora



Operaciones I

```
# Suma
2+2
```

```
4
```

```
# Resta
5-3
```

```
2
```

```
# Producto
3*5
```

```
15
```



Operaciones II

```
## Cuidado
2^2 # no funciona como potencia
```

```
0
```

```
### para la potenciación en python potencia se debe usar **
print(2**2)
```

```
4
```

```
9**(1/2)
```

```
3.0
```



Operaciones III

```
# El orden de las operaciones se altera con paréntesis

(4-2)*3**2
```

```
18
```

```
# Residuo de la división
7%4
```

```
3
```

```
# Parte entera de la división
7//4
```

```
1
```



Funciones I

Para tener disponibles funciones matemáticas es necesario importar el paquete `math`

```
# Raíz cuadrada
from math import sqrt
sqrt(9)
```

```
3.0
```

```
# exponencial
from math import exp
exp(1)
```

```
2.718281828459045
```

```
# logaritmo en base 10
from math import log10
log10(100)
```

```
2.0
```



Funciones II

```
from math import log
# logaritmo en base 10
log(100,10)
```

```
2.0
```

```
# logaritmo en base 2
log(8,2)
```

```
3.0
```



Funciones trigonométricas

Usando Python, calcular

- $\sin(\pi/2)$
- $\sin(\pi)$
- $\sin(2 \times \pi)$
- $\cos(\pi/2)$
- $\cos(\pi)$
- $\cos(2 \times \pi)$
- $\tan(\pi/4)$



Solución

```
from math import sin, cos , pi
```



Tuplas

Las tuplas sirven para agrupar objetos de diferente tipo, ellas estan *indexadas* y son **inmutables**, es decir, no se pueden modificar (agregar, quitar, cambiar entradas) una vez creadas.

```
mytupla0=('abc',4,True,5,False,2.5)
mytupla=(1,0.7,mytupla0,"Hola")
print(mytupla0)
```

```
('abc', 4, True, 5, False, 2.5)
```

```
mytupla
```

```
(1, 0.7, ('abc', 4, True, 5, False, 2.5), 'Hola')
```



Datos soportados por las tuplas

Importante

Note que una tupla puede contener distintos tipos de datos (string, enteros, lógicos y floats), incluso otras tuplas y otros tipos de contenedores



Acceso a datos en una tupla

Para acceder a los elementos de una tupla se usa `tupla[indice]` donde *indice* es un número entero de cero hasta la longitud de la tupla menos uno $n-1$

```
mytupla[0]                                                    ①
```

① Primer elemento de la tupla.

```
1
```



Ejecute I

```
mytupla[4]
mytupla[2]=0
mytupla[2]='xyz'
```



Ejecute II

```
mytupla[4] # IndexError: tuple index out of range
# no corre, a eso se refiere con inmutable
mytupla[2]=0 #TypeError: 'tuple' object does not support item assignment
mytupla[2]='xyz'
```



Observe

```
t1=(3,4)
print('t1= ',t1,'id de t1 antes ',id(t1))
```

```
t1=  (3, 4) id de t1 antes  4778509312
```

```
t2=(5,6)
print('t2= ',t2,'id de t2 antes ',id(t2))
```

```
t2=  (5, 6) id de t2 antes  4778496832
```

```
print(t2)
```

```
(5, 6)
```

```
t1=t1+t2
print('t1= ', t1,'id de t1 despues ', id(t1))
```

```
t1=  (3, 4, 5, 6) id de t1 despues  4778839392
```

```
print('t2= ', t2, 'id de t2 despues ', id(t2))
```



Ventajas de la inmutabilidad

- Seguridad: Como los datos no pueden ser modificados accidentalmente, es más seguro usar tuplas cuando se quiere proteger la integridad de los datos.
- Uso como claves en diccionarios: Las tuplas, al ser inmutables, pueden ser utilizadas como claves en diccionarios, algo que no es posible con las listas.
- Optimización: Las tuplas pueden ser más eficientes en términos de rendimiento y uso de memoria en comparación con las listas. La inmutabilidad hace que las tuplas sean útiles en situaciones donde se requiere un conjunto de datos constante y seguro.



Función type

Para averiguar la **clase** de un objeto usamos la función `type()`

```
type(mytupla)
```

```
<class 'tuple'>
```



Averiguar si un objeto es una tupla

- En algunas aplicaciones es necesario verificar si un objeto pertenece o no a una clase.
- Por ejemplo, para dirigir el flujo de ejecución del código de programación de una función: si el objeto entregado es una tupla, ejecute ciertas líneas de código, de lo contrario ejecute otras líneas.
- La función `isinstance(objeto,tipo)` regresa `True` si la clase del `objeto` entregado corresponde a `tipo`, en caso contrario regresa `False`.
- Otra forma es mediante la función `type`



Función isinstance

```
isinstance(mytupla, tuple)
```

```
True
```

```
isinstance('abcd', tuple)
```

```
False
```

```
isinstance('abcd', str)
```

```
True
```



Función isinstance

```
type(mytupla)
```

```
<class 'tuple'>
```

```
type(mytupla) is tuple
```

```
True
```

```
type(4) is tuple
```

```
False
```

```
type(4) is int
```

```
True
```



Otras formas de crear tuplas

Otra forma de crear tuplas es mediante la siguiente orden, con la segunda linea se imprime el tipo de objeto que es t3, con el fin de verificar que efectivamente se ha creado una tupla.

```
t3=10,20
type(t3)
```

```
<class 'tuple'>
```

Esta forma se conoce como creación de tuplas mediante *empaquetado*



Desempaquetado de tuplas

También podemos recuperar los elementos de una tupla mediante el *desempaquetado*, de la siguiente manera

```
x,y=t3
print("x= ",x)
print("y= ",y)
```

el primer elemento de la tupla `t3` se asigna al objeto `x` y el segundo a `y` luego se imprimen sus valores.



Tuplas especiales

Se pueden crear tuplas vacías y tuplas que contiene un solo valor, estas últimas llamadas tuplas unitarias, como se muestra a continuación

```
tupvacia=() # crea una tupla vacía
tupunit=(4,) # crea una tupla con un único elemento
print(tupvacia,type(tupvacia),tupunit,type(tupunit))
```

```
() <class 'tuple'> (4,) <class 'tuple'>
```

note el uso de la coma después del elemento de la tupla unitaria, eso debe ser así porque de lo contrario no se estaría creando una tupla, sino un objeto de tipo entero (`int`)

```
x=(4) # cuidado!!! x es un entero, no una tupla
print(x,type(x))
```

```
4 <class 'int'>
```



Indexado de tuplas.

Se puede crear una tupla a partir de otra mediante órdenes de la forma `tupla[i:j]` esto se conoce como indexación, veamos algunos ejemplos:

```
a=(3,2,4,1,6,4) # se crea la tupla a
b=a[1:4] # tupla b, creada a partir de a
print(b,type(b))
```

```
(2, 4, 1) <class 'tuple'>
```

Note en la salida anterior que la tupla `b` inicia en 2, y no en 3, que es el primer elemento de la tupla `a` ¿por qué?

```
a[:4]    # los cuatro primeros elementos de la tupla a
a[2:]    # desde el tercer elemento hasta el final
a[::-1]  # la tupla en reversa
```



Funciones y métodos aplicables a tuplas



Funciones y métodos aplicables a tuplas



La función `len()`

- La función `len()` regresa el número de elementos que contiene una tupla

```
print(len(t1)) # Imprime  la longitud  de la tupla
```

```
4
```

```
t1[len(t1)-1]  # recupera el último
```

```
6
```



Método .index(x)

- El método .index(x) regresa el índice que le corresponde al elemento x en la tupla, si el elemento está más de una vez regresa el índice de la primera ocurrencia, en caso que x no esté ocurre un error.

```
t1.index(4)     # índice correspondiente al primer 4 en t1
```

```
1
```



Método .count()

- El método .count() regresa el número de veces que un elemento está en la tupla, en caso de no estar regresa cero, a continuación se muestra como usarlos sobre la tupla t1 creada arriba

```
t1.count(4) # número de veces que está 4
```

```
1
```



! Importante

Habrá notado la diferencia en el llamado a `len()` frente a `.index()` y `.count()`:

- `len()` es una función genérica de Python, así que puede aplicarse a diferentes tipos de objetos,
- Mientras que los dos últimos son *métodos*, definidos sobre la clase *tuple*.



Diferencia entre método y función

- Cuando se crea una tupla se le asocian los métodos propios de la clase, es decir, el método está asociado al objeto.
- Las funciones sólo se pueden llamar por su nombre, ya que se definen de forma independiente,
- Los métodos no pueden ser llamados únicamente por su nombre, necesitamos invocar la clase por una referencia de esa clase en la que está definida.
- Esa es la razón por la que **no usamos** `index(t1,4)` o `count(t1,4)` sino `t1.index(4)` o `t1.count(4)`
- Porque estos son métodos definidos sobre la clase `tuple`, mientras que `len()` es una función a la cual se le entrega una tupla y regresa su longitud,
- `len()` se puede usar con otros tipos de contenedores, incluso con cadenas de caracteres.



Operaciones con tuplas



Operador `in`

Para saber si un elemento hace parte de una tupla, usamos el operador `in` el cual regresa un valor lógico `True` o `False` dependiendo de si el elemento se encuentra o no en la tupla.

```
4 in t1      # True sí 4 está en t1, False si no
```

```
True
```

```
"x" in t1    # True sí  "x" está en t1,  False si no
```

```
False
```

```
"abc" in t1 # True sí "abc" está en t1,  False si no
```

```
False
```



Concatenación de tuplas.

Se pueden concatenar y replicar tuplas usando los operadores + y * de la siguiente forma

```
Trats=("T1","T2","T3")
(2,3,4) + Trats # Concatena la tupla (2,3,4) con Trats
```

```
(2, 3, 4, 'T1', 'T2', 'T3')
```

```
Trats*3          # replica la tupla Trats tres veces
```

```
('T1', 'T2', 'T3', 'T1', 'T2', 'T3', 'T1', 'T2', 'T3')
```

```
(2,3,4)*2        # replica la tupla (2,3,4) tres veces
```

```
(2, 3, 4, 2, 3, 4)
```



Iterar sobre una tupla.

Una tupla es un objeto *iterable*, es decir, se pueden recorrer uno tras otro sus elementos usando bucles, a continuación un ejemplo usando `for`

```
for i in Trats:
    print(i)
```

```
T1
T2
T3
```



Aplicación

Escribir código que tome una tupla y nos informe cuántos enteros y no enteros contiene.

```
t1=('abc',4,True,5,False,2.5)
entero = 0
for i in t1:
    if type(i) is int:
        entero+=1
NoEnteros=len(t1)-entero
print(f"Hay {entero} enteros en la tupla y {NoEnteros} que no son enteros"
```

```
Hay 2 enteros en la tupla y 4 que no son enteros
```



Ejemplo de aplicación.

En el siguiente bloque de código se generan datos de una población normal con media cero y desviación estándar 10. Luego se usa la función `shapiro` del módulo `scipy.stats`, para llevar a cabo la prueba de *Shapiro Wilks* usando estos datos. Lo que se quiere resaltar es que la función regresa una tupla que tiene como primer elemento el estadístico W y en la segunda posición el $p-$valor de la prueba, veamos



Ejemplo de aplicación II

```
1 import scipy.stats as s                    ①
2 import numpy as np                         ②
3 np.random.seed(seed=123)                   ③
4 n = s.norm(0,10)                           ④
5 datos=n.rvs(100)                           ⑤
6 res=s.shapiro(datos)                       ⑥
```

① Carga del modulo `scipy.stats`.
② Carga del paquete `numpy`
③ Establece una semilla para el generador de números aleatorios de `NumPy`, para que el ejemplo sea reproducible.
④ Se `instancia` una variable aleatoria normal de media 0 y desviación estándar 10.
⑤ Se simula la toma de una muestra aleatoria de una población con distribución normal de media 0 y desviación estándar 10.
⑥ Prueba de Shapiro Wilks.



Ejemplo de aplicación III

Podemos recuperar los valores del estadístico de prueba, W, y el $p-$valor, mediante el desempaquetado o mediante el indexado como se muestra a continuación

```
W,p=res  # desempaquetado, equivalente a W=res[0] ; p=res[1]
```

si estamos escribiendo un informe donde sea necesario reportar el resultado de la prueba podríamos usar

```
print("Estadístico de S-Wilks, %0.3f, p_valor = %0.4f" %res)
```



¿Cómo saber si un objeto de Python es una tupla?

En algunas aplicaciones es necesario verificar si un objeto pertenece o no a una clase. Por ejemplo, para dirigir el flujo de ejecución del código de programación de una función: si el objeto entregado es una tupla, ejecute ciertas líneas de código, de lo contrario ejecute otras líneas. La función `isinstance(objeto,tipo)` regresa `True` si la clase del `objeto` entregado corresponde a `tipo`, en caso contrario regresa `False`.

```
isinstance(t1, tuple)
```

```
True
```



Función `type()`

Otra forma es usando la función `type()`, como se muestra a continuación

```
type(t1) is tuple
```

```
True
```

```
type(4) is tuple
```

```
False
```



Otros tipos de datos

Esta última línea regresa `False` porque el número `4` no es una tupla. En las siguientes líneas se ilustra como averiguar por los tipos de datos más usados en estadística

```
type(x) is int    # True si x es entero, False en otro caso
type(x) is float  # True si x un real, False en otro caso
type(x) is str    # True si x es string,  False en otro caso
type(x) is bool   # True si x es lógico,  False en otro caso
```



Ejemplo

A manera de ejemplo, programar una función que reciba un objeto y en caso que este sea una tupla regrese el mensaje `'Es una tupla'`, en caso contrario regrese `'No es una tupla'`

```
1 # Se define la función
2 def verificartupla(x):
3   if(type(x) is tuple):
4     print("Es una tupla")
5   else:
6     print("No es una tupla")
7 # Fin de definición de función
```



Uso de la función

```
# Uso de  la función
verificartupla(t1) # Retorna: `True`, `t1` es una tupla
verificartupla("hola") # Retorna: `False`, `"hola"` no es una tupla
```

